

EDS Lab Assignments

Practical4

Name: Prasad Devidas Devkar.

PRN: 202501040056

4.1.1 Pandas – Series Creation and Manipulation

Problem Statement:

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Code:

```
import pandas as pd

# Take inputs from the user to create a list of numbers
numbers = list(map(int, input().split()))

# Create a Pandas series from the list of numbers
series = pd.Series(numbers)

# Grouping by even and odd numbers and calculating the mean
grouped = series.groupby(series % 2 == 0).mean()

# Display the mean of even and odd numbers with labels
grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
print("Mean of even and odd numbers:")
print(grouped)
```

Output:

Average time
0.053 s
53.17 ms

Maximum time
0.074 s
74.00 ms

3 out of 3 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 156 msDebug

Expected output

1 2 3 4 5 6 7 8 9 10

Mean of even and odd numbers:

Odd...5.0

Even...6.0

dtype: float64

Actual output

1 2 3 4 5 6 7 8 9 10

Mean of even and odd numbers:

Odd...5.0

Even...6.0

dtype: float64

Test case 274 msDebug

Expected output

4 4 4 6 6 2 2 2 10 12 14 16

Mean of even and odd numbers:

Even...6.833333

dtype: float64

Actual output

4 4 4 6 6 2 2 2 10 12 14 16

Mean of even and odd numbers:

Even...6.833333

dtype: float64

Test case 368 msDebug

Expected output

1 5 7 9 11 13 15 17 111 21 25

Mean of even and odd numbers:

Odd...21.363636

dtype: float64

Actual output

1 5 7 9 11 13 15 17 111 21 25

Mean of even and odd numbers:

Odd...21.363636

dtype: float64

4.1.2 Dictionary to Dataframe

Problem Statement:

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the **Age** column.
- Display the DataFrame after deleting the column.

Code:

```
import pandas as pd

# Provided dictionary of lists
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
}

# Convert the dictionary to a DataFrame
df = pd.DataFrame(data)
```

```
# Display the original DataFrame
print("Original DataFrame:")
print(df)
```

```
# Adding a new
Name = input("New name: ")
Age = int(input("New age: "))
new = {'Name':Name, 'Age':Age}
df.loc[len(df)] = new
# Display the DataFrame after adding a new row
print("After adding a row:\n",df)
```

```
# Modifying a row
index = int(input("Index of row to modify: "))
age = int(input("New age: "))
df.at[index, 'Age']=age
# Display the DataFrame after modifying a row
print("After modifying a row:")
print(df)
```

```
# Deleting a row
delt = int(input("Index of row to delete: "))
df = df.drop(delt).reset_index(drop=True)
# Display the DataFrame after deleting a row
print("After deleting a row:")
print(df)
```

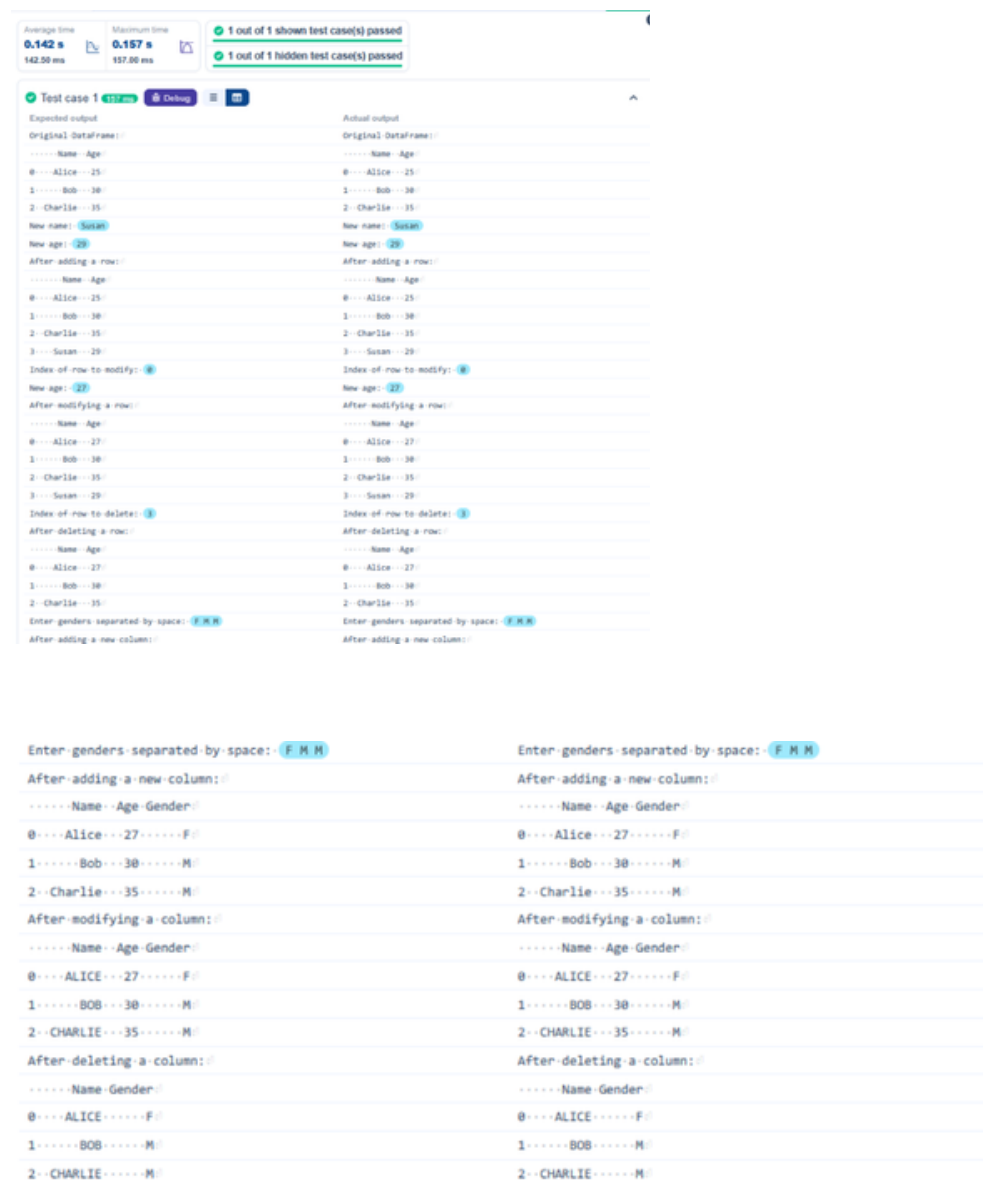
```
# Adding a new column
gen = input("Enter genders separated by space: ").split()
df['Gender']=gen
# Display the DataFrame after adding a new column
print("After adding a new column:")
print(df)
```

```
# Modifying a column
df['Name']=df['Name'].str.upper()
# Display the DataFrame after modifying a column
print("After modifying a column:")
print(df)
```

```
# Deleting a column
df = df.drop(columns = ['Age'])\
```

```
# Display the DataFrame after deleting a column
print("After deleting a column:")
print(df)
```

Output:



The screenshot displays a Jupyter Notebook interface with a test runner at the top showing '1 out of 1 shown test case(s) passed' and '1 out of 1 hidden test case(s) passed'. Below this, the 'Test case 1' section shows a comparison between 'Expected output' and 'Actual output' for a series of data manipulation tasks. The tasks include adding a new row, modifying a row, deleting a row, and adding a new column. The final output shows the DataFrame after deleting a column, with the 'Name' and 'Age' columns remaining.

Task	Expected Output	Actual Output
Original DataFrame	<pre>Name Age 0 Alice 25 1 Bob 30 2 Charlie 35 </pre>	<pre>Name Age 0 Alice 25 1 Bob 30 2 Charlie 35 </pre>
After adding a row	<pre>Name Age 0 Alice 25 1 Bob 30 2 Charlie 35 3 Susan 29 </pre>	<pre>Name Age 0 Alice 25 1 Bob 30 2 Charlie 35 3 Susan 29 </pre>
After modifying a row	<pre>Name Age 0 Alice 27 1 Bob 30 2 Charlie 35 3 Susan 29 </pre>	<pre>Name Age 0 Alice 27 1 Bob 30 2 Charlie 35 3 Susan 29 </pre>
After deleting a row	<pre>Name Age 0 Alice 27 1 Bob 30 2 Charlie 35 </pre>	<pre>Name Age 0 Alice 27 1 Bob 30 2 Charlie 35 </pre>
After adding a new column	<pre>Name Age Gender 0 Alice 27 F 1 Bob 30 M 2 Charlie 35 M </pre>	<pre>Name Age Gender 0 Alice 27 F 1 Bob 30 M 2 Charlie 35 M </pre>

4.1.3 Student Information

Problem Statement:

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Code:

```
import pandas as pd
# Read the text file into a DataFrame
file = input()
data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])
# write your code here..
print("First five rows:")
print(data.head())
average_age = round(data["Age"].mean(), 2)
print(f"Average age: {average_age}")
filtered_students = data[data["Grade"] <= "B"]
print("Students with a grade up to B")
print(filtered_students)
```

Output:

The screenshot shows a code execution environment with the following details:

- Performance Metrics:**
 - Average time: 0.042 s (41.50 ms)
 - Maximum time: 0.049 s (49.00 ms)
- Test Results:**
 - 1 out of 1 shown test case(s) passed
 - 1 out of 1 hidden test case(s) passed
- Test Case 1 (49 ms):**
 - Expected output:**

```
studentdata.txt
First-five-rows:
--Name--Age--Grade--
0--John--25-----A--
1--Alin--22-----B--
2--Emma--24-----A--
3--Mike--23-----C--
4--Sony--21-----B--
Average-age: 23.29
Students-with-a-grade-up-to-B:
--Name--Age--Grade--
0--John--25-----A--
1--Alin--22-----B--
2--Emma--24-----A--
4--Sony--21-----B--
5--Amia--26-----A--
```
 - Actual output:**

```
studentdata.txt
First-five-rows:
--Name--Age--Grade--
0--John--25-----A--
1--Alin--22-----B--
2--Emma--24-----A--
3--Mike--23-----C--
4--Sony--21-----B--
Average-age: 23.29
Students-with-a-grade-up-to-B:
--Name--Age--Grade--
0--John--25-----A--
1--Alin--22-----B--
2--Emma--24-----A--
4--Sony--21-----B--
5--Amia--26-----A--
```

4.2.1 Month with the Highest Total Sales

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.

- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Code:

```
import pandas as pd
# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
df['Date'] = pd.to_datetime(df['Date'])
df['Total_Sales_Amount'] = df['Quantity'] * df['Price']
Monthly_Sales = df.groupby(df['Date'].dt.strftime('%Y-%m'))
['Total_Sales_Amount'].sum()

# Find the month with the highest total sales
best_month = Monthly_Sales.idxmax()
highest_sales = Monthly_Sales.max()
print(f"Best month: {best_month}")
print(f"Total sales: ${highest_sales:.2f}")
```

Output:



The screenshot shows a code execution interface with the following details:

- Performance Metrics:**
 - Average time: 0.033 s (33.00 ms)
 - Maximum time: 0.038 s (38.00 ms)
- Test Results:**
 - 1 out of 1 shown test case(s) passed
 - 2 out of 2 hidden test case(s) passed
- Test Case 1 Details:**
 - Duration: 38 ms
 - Buttons: Debug, Run, Stop
- Expected vs Actual Output:**

Expected output	Actual output
sales_data.csv	sales_data.csv
Best month: 2025-01	Best month: 2025-01
Total sales: \$1210.00	Total sales: \$1210.00

4.2.2 Best Selling Product

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.

- Display the product that sold the most and the total quantity sold for that product.

Code:

```
import pandas as pd

# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
product_sales = df.groupby("Product")["Quantity"].sum()
# Find the product with the highest total quantity sold
best_product = product_sales.idxmax()
highest_quantity = product_sales.max()
# Display the result
print(f"Best selling product: {best_product}")
print(f"Total quantity sold: {highest_quantity}")
```

Output:



The screenshot shows a code execution interface with the following details:

- Performance Metrics:**
 - Average time: 0.025 s (25.00 ms)
 - Maximum time: 0.028 s (28.00 ms)
- Test Results:**
 - 1 out of 1 shown test case(s) passed
 - 2 out of 2 hidden test case(s) passed
- Test Case 1 Details:**
 - Status: Passed (27 ms)
 - Buttons: Debug, Menu, Close
- Expected vs Actual Output Comparison:**

Expected output	Actual output
sales_data.csv	sales_data.csv
Best-selling product: Product A	Best-selling product: Product A
Total quantity sold: 23	Total quantity sold: 23

4.2.3 City that Sold the Most Products

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.

- Find the city that sold the most products (based on the total quantity sold).

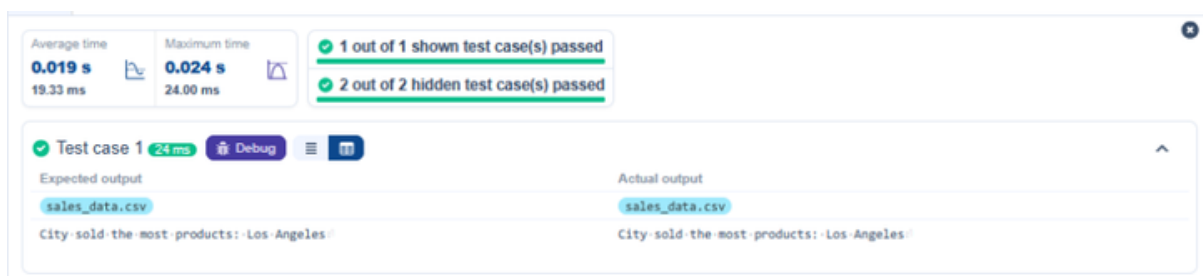
Code:

```
import pandas as pd

# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
# write the code..
city_sales = df.groupby("City")["Quantity"].sum()
best_city = city_sales.idxmax()

# Display the result
print(f"City sold the most products: {best_city}")
```

Output:



The screenshot shows a code execution interface with the following details:

- Performance Metrics:**
 - Average time: 0.019 s (19.33 ms)
 - Maximum time: 0.024 s (24.00 ms)
- Test Results:**
 - 1 out of 1 shown test case(s) passed
 - 2 out of 2 hidden test case(s) passed
- Test Case 1:**
 - Duration: 24 ms
 - Buttons: Debug, Run, Stop
- Expected output:**
 - sales_data.csv
 - City sold the most products: Los Angeles
- Actual output:**
 - sales_data.csv
 - City sold the most products: Los Angeles

4.2.4 Most Frequently Sold Product Pairs

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Code:

```
import pandas as pd

from itertools import combinations
from collections import Counter

# Prompt user to input the file name
file_name = input()

# Read data from the specified CSV file
df = pd.read_csv(file_name)

# write the code
grouped = df.groupby("Date")["Product"].apply(list)

pairs_counter = Counter()
for products in grouped:
    pairs = combinations(sorted(products), 2)
    pairs_counter.update(pairs)

max_frequency = max(pairs_counter.values())
most_common_pairs = [(pair, freq) for pair, freq in pairs_counter.items() if freq ==
max_frequency]

# Output the most frequent product pairs
for(product1, product2), frequency in most_common_pairs:
    print(f"{product1} and {product2}: {frequency} times")
```

Output:

The screenshot displays a code execution interface with a status bar at the top and a detailed test case view below. The status bar shows an average time of 0.020 s (19.67 ms) and a maximum time of 0.020 s (20.00 ms). It also indicates that 1 out of 1 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. The detailed view for 'Test case 1' (20 ms) shows a 'Debug' button and a comparison between 'Expected output' and 'Actual output'. Both outputs are identical, showing 'sales_data.csv' followed by two lines of product pairs and their frequencies: 'Product A and Product B: 2 times' and 'Product A and Product C: 2 times'.

Test Case	Status	Time	Expected Output	Actual Output
Test case 1	Passed	20 ms	sales_data.csv Product A and Product B: 2 times Product A and Product C: 2 times	sales_data.csv Product A and Product B: 2 times Product A and Product C: 2 times

4.2.5 Titanic Dataset Analysis and Data Cleaning

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.

6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (mode()).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

Code:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
print(data.head())
```

```
# 1. Display the first 5 rows of the dataset
print(data.tail())
```

```
# 2. Display the last 5 rows of the dataset
print(data.shape)
```

```
# 3. Get the shape of the dataset
print(data.info())
```

```
# 4. Get a summary of the dataset (info)
print(data.describe())
```

```
# 5. Get basic statistics of the dataset
print(data.isnull().sum())
```

```
# 6. Check for missing value
median_age = data['Age'].median()
data['Age'].fillna(median_age, inplace = True)
```

```
# 7. Fill missing values in the 'Age' column with the median age
mode_embarked = data['Embarked'].mode() [0]
data['Embarked'].fillna(mode_embarked, inplace = True)
```

```
# 8. Fill missing values in the 'Embarked' column with the mode
data.drop('Cabin', axis=1, inplace = True)
```

9. Drop the 'Cabin' column due to many missing values

`data['FamilySize'] = data['SibSp'] + data['Parch']`

10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'

Output:

Average time 0.185 s 185.00 ms		Maximum time 0.185 s 185.00 ms	1 out of 1 shown test case(s) passed
Test case 1 185 ms Debug			
Expected output		Actual output	
PassengerId Survived Pclass Fare Cabin Embarked		PassengerId Survived Pclass Fare Cabin Embarked	
0 1 0 7.2500 NaN S		0 1 0 7.2500 NaN S	
1 2 1 71.2833 C85 C		1 2 1 71.2833 C85 C	
2 3 1 7.9250 NaN S		2 3 1 7.9250 NaN S	
3 4 1 53.1000 C123 S		3 4 1 53.1000 C123 S	
4 5 0 8.0500 NaN S		4 5 0 8.0500 NaN S	
[5 rows x 12 columns]		[5 rows x 12 columns]	
PassengerId Survived Pclass Fare Cabin Embarked		PassengerId Survived Pclass Fare Cabin Embarked	
886 887 0 13.00 NaN S		886 887 0 13.00 NaN S	
887 888 1 30.00 B42 S		887 888 1 30.00 B42 S	
888 889 0 23.45 NaN S		888 889 0 23.45 NaN S	
889 890 1 30.00 C148 C		889 890 1 30.00 C148 C	
890 891 0 7.75 NaN Q		890 891 0 7.75 NaN Q	
[5 rows x 12 columns]		[5 rows x 12 columns]	
(891, 12)		(891, 12)	
<class 'pandas.core.frame.DataFrame'>		<class 'pandas.core.frame.DataFrame'>	
RangeIndex: 891 entries, 0 to 890		RangeIndex: 891 entries, 0 to 890	
Data columns (total 12 columns):		Data columns (total 12 columns):	
# Column Non-Null Count Dtype		# Column Non-Null Count Dtype	
0 PassengerId 891 non-null int64		0 PassengerId 891 non-null int64	
1 Survived 891 non-null int64		1 Survived 891 non-null int64	
2 Pclass 891 non-null int64		2 Pclass 891 non-null int64	
3 Name 891 non-null object		3 Name 891 non-null object	
4 Sex 891 non-null object		4 Sex 891 non-null object	
5 Age 714 non-null float64		5 Age 714 non-null float64	
6 SibSp 891 non-null int64		6 SibSp 891 non-null int64	
7 Parch 891 non-null int64		7 Parch 891 non-null int64	
8 Ticket 891 non-null object		8 Ticket 891 non-null object	
9 Fare 891 non-null float64		9 Fare 891 non-null float64	
10 Cabin 284 non-null object		10 Cabin 284 non-null object	

Average time 0.185 s 185.00 ms Maximum time 0.185 s 185.00 ms 1 out of 1 shown test case(s) passed	
-10- Cabin-----204 non-null----object-	-10- Cabin-----204 non-null----object-
-11- Embarked-----889 non-null----object-	-11- Embarked-----889 non-null----object-
dtypes: float64(2), int64(5), object(5)	dtypes: float64(2), int64(5), object(5)
memory usage: 66.2+ KB	memory usage: 66.2+ KB
None	None
-----PassengerId----Survived-----Pclass-----SibSp-----Parch	-----PassengerId----Survived-----Pclass-----SibSp-----Parch
-----Fare	-----Fare
count--891.000000--891.000000--891.000000--...,--891.000000--891.000000--891.000000	count--891.000000--891.000000--891.000000--...,--891.000000--891.000000--891.000000
mean--446.000000--0.383838--2.308642--...,--0.523008--0.381594--32.204208	mean--446.000000--0.383838--2.308642--...,--0.523008--0.381594--32.204208
std--257.353842--0.486592--0.836071--...,--1.102743--0.806057--49.693429	std--257.353842--0.486592--0.836071--...,--1.102743--0.806057--49.693429
min--1.000000--0.000000--1.000000--...,--0.000000--0.000000--0.000000	min--1.000000--0.000000--1.000000--...,--0.000000--0.000000--0.000000
25%--223.500000--0.000000--2.000000--...,--0.000000--0.000000--7.910400	25%--223.500000--0.000000--2.000000--...,--0.000000--0.000000--7.910400
50%--446.000000--0.000000--3.000000--...,--0.000000--0.000000--14.454200	50%--446.000000--0.000000--3.000000--...,--0.000000--0.000000--14.454200
75%--668.500000--1.000000--3.000000--...,--1.000000--0.000000--31.000000	75%--668.500000--1.000000--3.000000--...,--1.000000--0.000000--31.000000
max--891.000000--1.000000--3.000000--...,--8.000000--6.000000--512.329200	max--891.000000--1.000000--3.000000--...,--8.000000--6.000000--512.329200
[8 rows x 7 columns]	[8 rows x 7 columns]
PassengerId-----0	PassengerId-----0
Survived-----0	Survived-----0
Pclass-----0	Pclass-----0
Name-----0	Name-----0
Sex-----0	Sex-----0
Age-----177	Age-----177
SibSp-----0	SibSp-----0
Parch-----0	Parch-----0
Ticket-----0	Ticket-----0
Fare-----0	Fare-----0
Cabin-----687	Cabin-----687
Embarked-----2	Embarked-----2
dtype: int64	dtype: int64

4.2.6 Titanic Dataset Analysis and Data Cleaning-2

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

Code:

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']

# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
data['Alone'] = np.where(data['FamilySize'] == 0, 1, 0)

# 2. Convert 'Sex' to numeric (male: 0, female: 1)
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

# 3. One-hot encode the 'Embarked' column
data = pd.get_dummies(data, columns = ['Embarked'], drop_first = True)

# 4. Get the mean age of passengers
mean_age = data['Age'].mean()
print(mean_age)

# 5. Get the median fare of passengers
median_fare = data['Fare'].median()
print(median_fare)

# 6. Get the number of passengers by class
```

```
passengers_by_class = data['Pclass'].value_counts()
print(passengers_by_class)
```

```
# 7. Get the number of passengers by gender
passengers_by_gender = data['Sex'].value_counts().sort_index()
print(passengers_by_gender)
```

```
# 8. Get the number of passengers by survival status
passengers_by_survival = data['Survived'].value_counts().sort_index()
print(passengers_by_survival)
```

```
# 9. Calculate the survival rate
survival_rate = data['Survived'].mean()
print(survival_rate)
```

```
# 10. Calculate the survival rate by gender
survival_rate_by_gender = data.groupby('Sex')['Survived'].mean()
print(survival_rate_by_gender)
```

Output:

Average time
0.022 s
22.00 ms

Maximum time
0.022 s
22.00 ms

1 out of 1 shown test case(s) passed

Test case 1 22 ms Debug

Expected output	Actual output
29.69911764705882	29.69911764705882
14.4542	14.4542
3...491	3...491
1...216	1...216
2...184	2...184
Name: Pclass, dtype: int64	Name: Pclass, dtype: int64
0...577	0...577
1...314	1...314
Name: Sex, dtype: int64	Name: Sex, dtype: int64
0...549	0...549
1...342	1...342
Name: Survived, dtype: int64	Name: Survived, dtype: int64
0.3838383838383838	0.3838383838383838
Sex:	Sex:
0...0.188908	0...0.188908
1...0.742038	1...0.742038
Name: Survived, dtype: float64	Name: Survived, dtype: float64

4.2.7 Titanic Dataset Analysis and Data Cleaning-3

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

Code:

```
import pandas as pd
import numpy as np
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 1. Calculate the survival rate by class
print(data.groupby('Pclass')['Survived'].mean())
# 2. Calculate the survival rate by embarked location
print(data.groupby('Embarked_S')['Survived'].mean())
# 3. Calculate the survival rate by family size
print(data.groupby('FamilySize')['Survived'].mean())
# 4. Calculate the survival rate by being alone
print(data.groupby('IsAlone')['Survived'].mean())
# 5. Get the average fare by class
print(data.groupby('Pclass')['Fare'].mean())
# 6. Get the average age by class
print(data.groupby('Pclass')['Age'].mean())
# 7. Get the average age by survival status
print(data.groupby('Survived')['Age'].mean())
# 8. Get the average fare by survival status
print(data.groupby('Survived')['Fare'].mean())
# 9. Get the number of survivors by class
print(data[data['Survived'] == 1]['Pclass'].value_counts())
# 10. Get the number of non-survivors by class
```

```
print(data[data['Survived'] == 0]['Pclass'].value_counts())
```

Output:

Average time 0.057 s 57.00 ms Maximum time 0.057 s 57.00 ms 1 out of 1 shown test case(s) passed	
Test case 1 57 ms Debug	
Expected output	Actual output
Pclass	Pclass
1----0.629638	1----0.629638
2----0.472826	2----0.472826
3----0.242363	3----0.242363
Name: Survived, dtype: float64	Name: Survived, dtype: float64
Embarked_S	Embarked_S
0----0.506073	0----0.506073
1----0.336957	1----0.336957
Name: Survived, dtype: float64	Name: Survived, dtype: float64
FamilySize	FamilySize
0----0.303538	0----0.303538
1----0.552795	1----0.552795
2----0.578431	2----0.578431
3----0.724138	3----0.724138
4----0.200000	4----0.200000
5----0.136364	5----0.136364
6----0.333333	6----0.333333
7----0.000000	7----0.000000
10----0.000000	10----0.000000
Name: Survived, dtype: float64	Name: Survived, dtype: float64
IsAlone	IsAlone
0----0.505650	0----0.505650
1----0.303538	1----0.303538
Name: Survived, dtype: float64	Name: Survived, dtype: float64
Pclass	Pclass
1----84.154687	1----84.154687
2----20.662183	2----20.662183
3----13.675550	3----13.675550
Name: Fare, dtype: float64	Name: Fare, dtype: float64
Pclass	Pclass
1----38.233441	1----38.233441
2----29.877630	2----29.877630
3----25.140620	3----25.140620
Name: Age, dtype: float64	Name: Age, dtype: float64
Survived	Survived
0----30.626179	0----30.626179
1----28.343090	1----28.343090
Name: Age, dtype: float64	Name: Age, dtype: float64
Survived	Survived
0----22.117887	0----22.117887
1----48.395408	1----48.395408
Name: Fare, dtype: float64	Name: Fare, dtype: float64
1----136	1----136
3----119	3----119
2----87	2----87
Name: Pclass, dtype: int64	Name: Pclass, dtype: int64
3----372	3----372
2----97	2----97
1----80	1----80
Name: Pclass, dtype: int64	Name: Pclass, dtype: int64

4.2.8 Titanic Dataset Analysis and Data Cleaning-4

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

Code:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
# 1. Get the number of survivors by gender
survivors_by_gender = data[data['Survived'] == 1]['Sex'].value_counts()
print(survivors_by_gender)
```

```
# 2. Get the number of non-survivors by gender
non_survivors_by_gender = data[data['Survived'] == 0]['Sex'].value_counts()
print(non_survivors_by_gender)
```

```
# 3. Get the number of survivors by embarked location
survivors_by_embarked_s = data[data['Survived'] == 1]
['Embarked_S'].value_counts()
print(survivors_by_embarked_s)
```

```
# 4. Get the number of non-survivors by embarked location
non_survivors_by_embarked_s = data[data['Survived'] == 0]
['Embarked_S'].value_counts()
print(non_survivors_by_embarked_s)
```

```
# 5. Calculate the percentage of children (Age < 18) who survived
children = data[data['Age'] < 18]
children_survival_rate = children['Survived'].mean()
print(children_survival_rate)
```

6. Calculate the percentage of adults (Age >= 18) who survived

```
adults = data[data['Age'] >= 18]
adults_survival_rate = adults['Survived'].mean()
print(adults_survival_rate)
```

7. Get the median age of survivors

```
median_age_survivors = data[data['Survived'] == 1]['Age'].median()
print(median_age_survivors)
```

8. Get the median age of non-survivors

```
median_age_non_survivors = data[data['Survived'] == 0]['Age'].median()
print(median_age_non_survivors)
```

9. Get the median fare of survivors

```
median_fare_survivors = data[data['Survived'] == 1]['Fare'].median()
print(median_fare_survivors)
```

10. Get the median fare of non-survivors

```
median_fare_non_survivors = data[data['Survived'] == 0]['Fare'].median()
print(median_fare_non_survivors)
```

Output:

Average time

0.030 s
30.00 ms

Maximum time

0.030 s
30.00 ms

1 out of 1 shown test case(s) passed

Test case 1 30 ms

Debug



Expected output

Actual output

female...233

female...233

male...109

male...109

Name: Sex, dtype: int64

Name: Sex, dtype: int64

male...468

male...468

female...81

female...81

Name: Sex, dtype: int64

Name: Sex, dtype: int64

1...217

1...217

0...125

0...125

Name: Embarked_S, dtype: int64

Name: Embarked_S, dtype: int64

1...427

1...427

0...122

0...122

Name: Embarked_S, dtype: int64

Name: Embarked_S, dtype: int64

0.5398230088495575

0.5398230088495575

0.3810316139767055

0.3810316139767055

28.0

28.0

28.0

28.0

26.0

26.0

10.5

10.5